Project Learning Guide v2

Real-Time OV7670 Video Capture and VGA Processing System

Basys 3 Artix-7 FPGA, OV7670 camera, dual-port frame buffer, VGA output

Why this version exists. The first PDF used many LaTeX-drawn charts that were hard to read. This v2 guide replaces those charts with large standalone PNG image diagrams. Each major idea has its own full-width picture.

How to read this guide. Start with the architecture image, then follow the images in order: startup, clock domains, camera capture, frame buffer, VGA timing, display mapping, filters, and debug flow.

Project goal	Capture live OV7670 video, store it in BRAM, process it, and output VGA.
Stored frame	320 x 240 RGB565 pixels.
VGA output	640 x 480 timing at about 59.5 Hz using a 25 MHz pixel clock.
Main safety rule	Camera and VGA use different clocks, so pixel data crosses through dual-port BRAM.

1. System Architecture

The design has two major paths. The configuration path writes OV7670 registers using SCCB before live video is useful. The video path captures bytes, writes BRAM, reads BRAM, scales/maps the image, applies a filter, and outputs VGA.

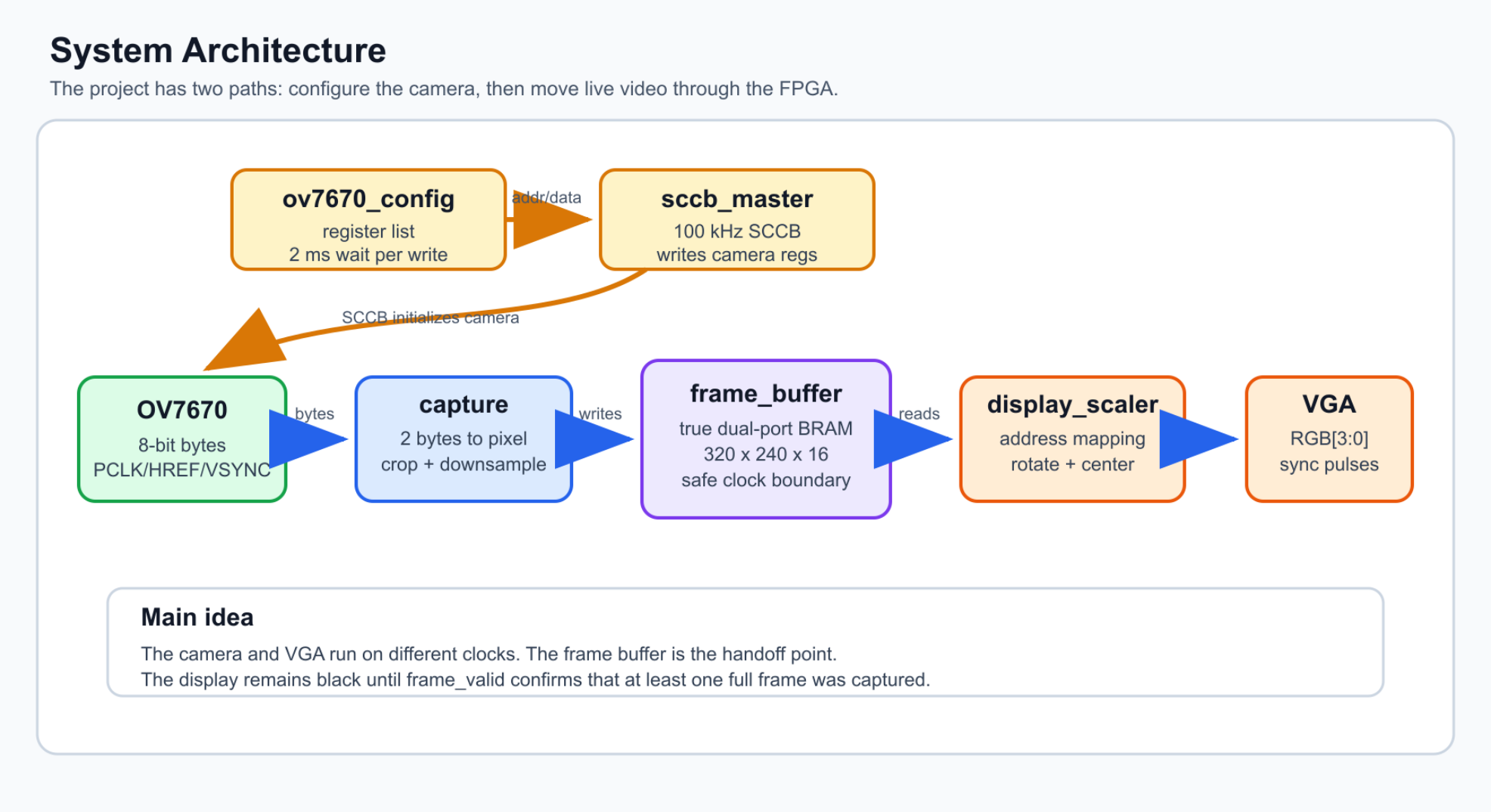

Figure 1: Readable system architecture diagram generated as an image.

2. Startup and Camera Configuration

The OV7670 must be configured before useful image data appears. `ov7670_config.v` provides register address/data pairs. `sccb_master.v` turns each pair into an SCCB write.

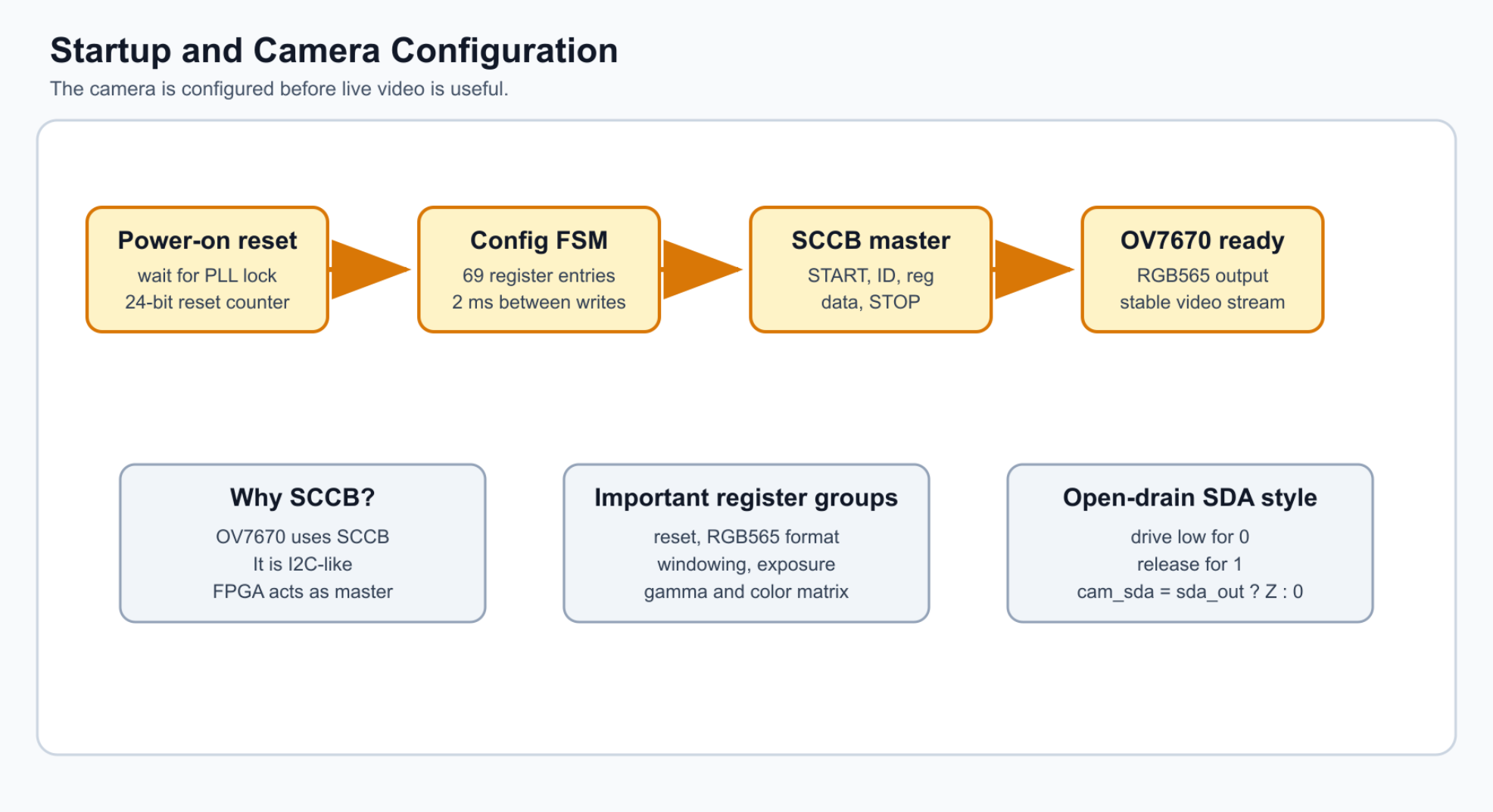

Figure 2: Startup and OV7670 SCCB configuration flow.

Only these register groups matter for understanding the project: soft reset, clocking, RGB565 format, image windowing, exposure/gain/white-balance, gamma, color matrix, and stability settings.

3. Clock Domains and CDC

This project is not a single-clock design. The system clock, camera pixel clock, and VGA pixel clock are separate timing domains.

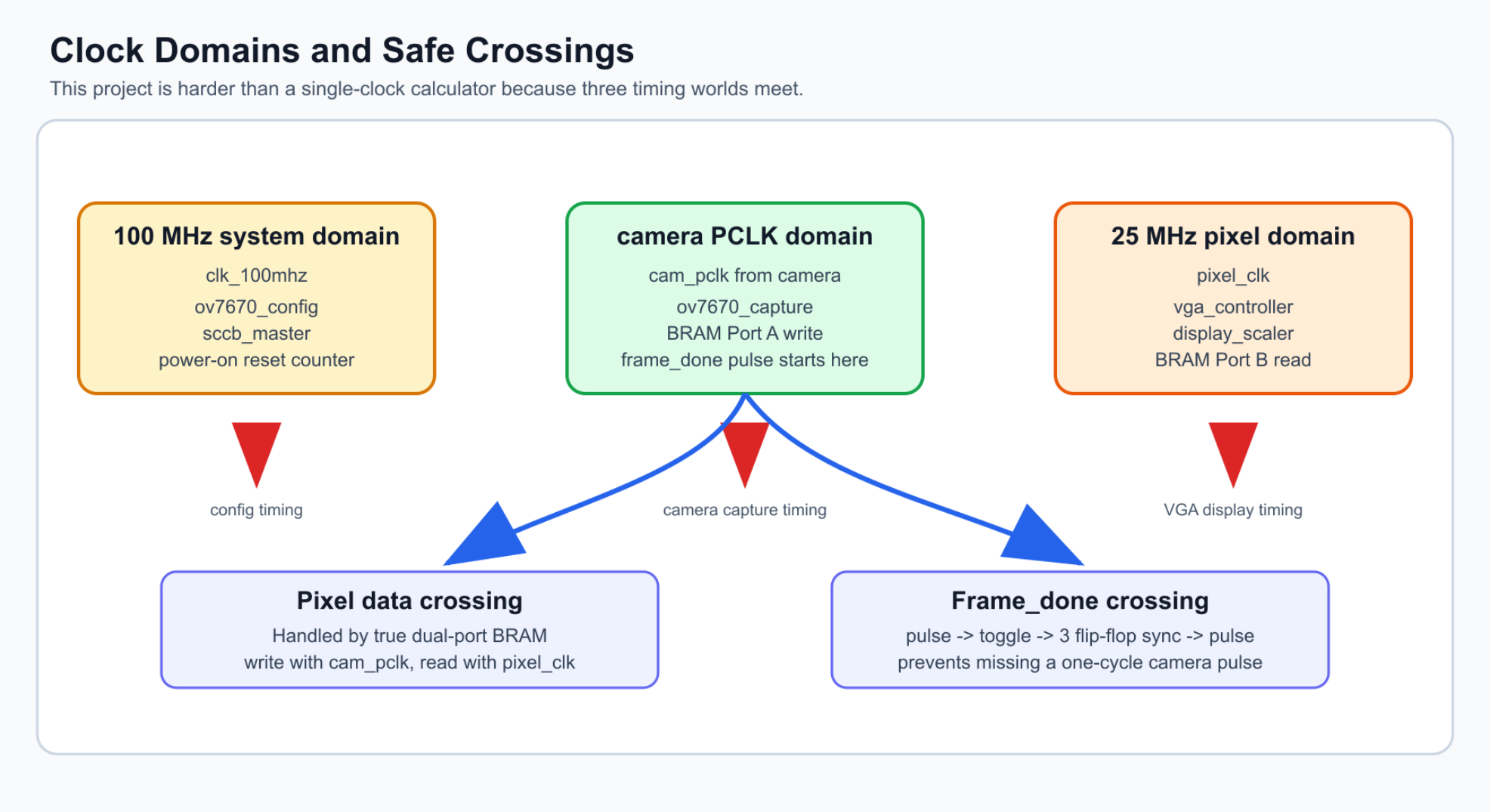

Figure 3: Clock domains and safe crossing methods.

Clock	Domain	Used by
`clk_100mhz`	100 MHz system	SCCB configuration and reset counter.
`cam_pclk`	camera output clock	Camera byte capture and BRAM Port A writes.
`pixel_clk`	25 MHz VGA clock	VGA timing, display mapping, BRAM Port B reads.

4. Camera Capture

The OV7670 sends one byte at a time. `ov7670_capture.v` saves the first byte, combines it with the second byte, crops unstable edge pixels, downsamples, and writes one RGB565 pixel into BRAM.

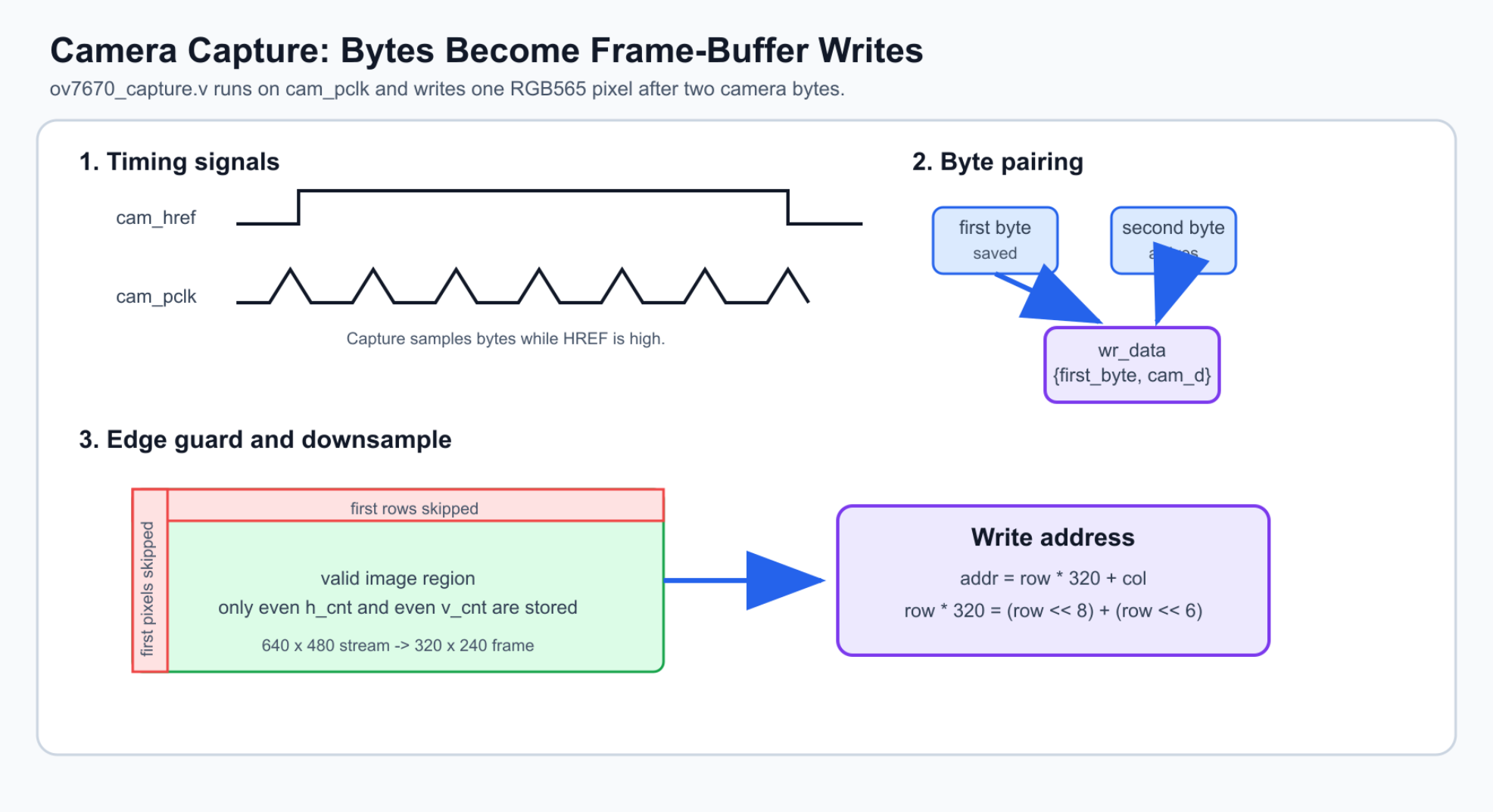

Figure 4: Camera byte stream converted into frame-buffer writes.

The important code idea is:

```
wr_data <= {first_byte, cam_d};
wr_en   <= 1'b1;
wr_addr <= row * 320 + col;
```

The RTL avoids a direct multiply by 320 by using:

```
row * 320 = (row << 8) + (row << 6)
```

5. Frame Buffer BRAM

The frame buffer is the center of the design. It decouples the camera clock from the VGA clock.

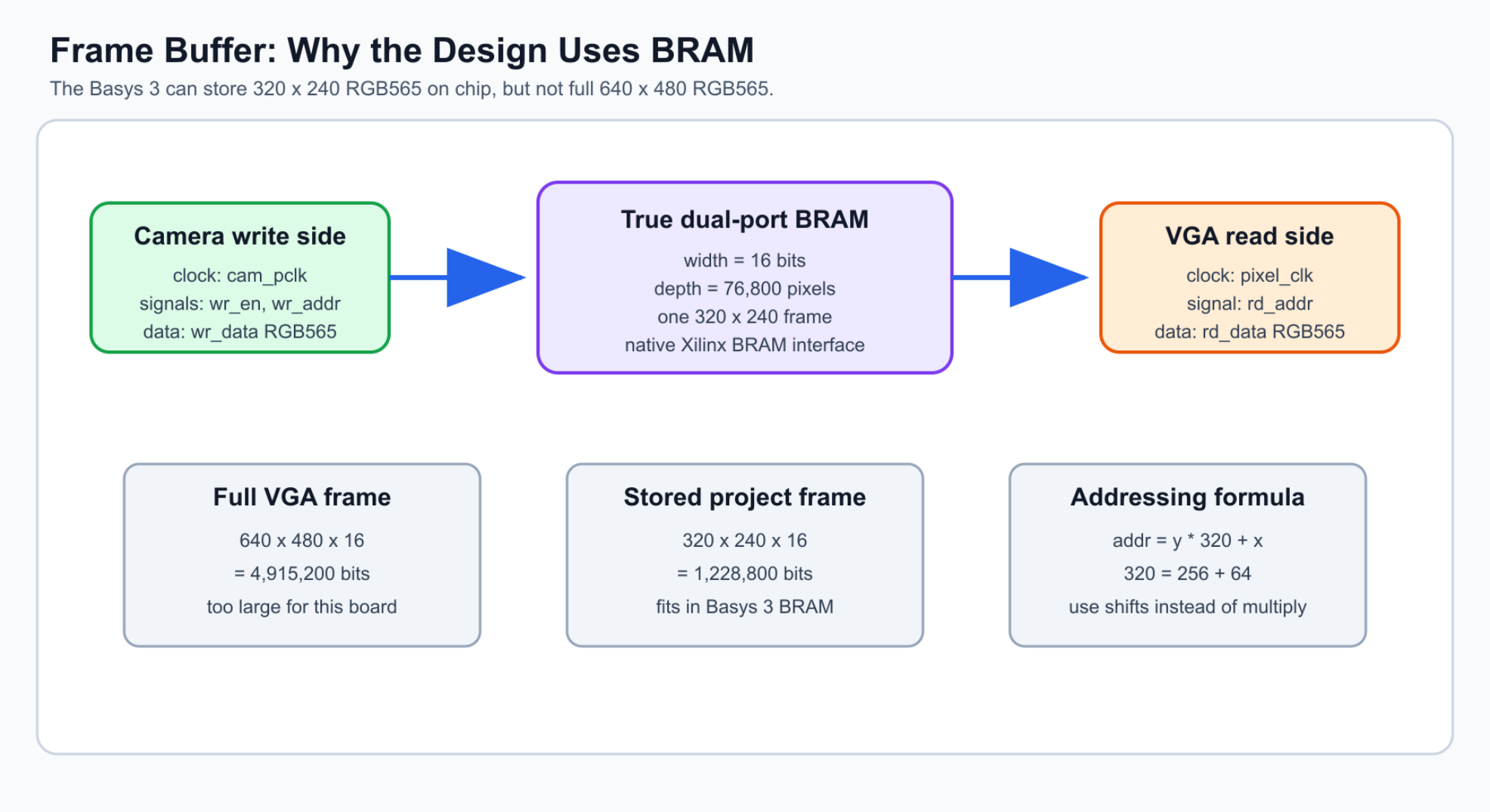

Figure 5: Dual-port BRAM frame buffer and memory-size reasoning.

Why 320 x 240? A full 640 x 480 RGB565 frame needs 4,915,200 bits, which is too large for this Basys 3 design. A 320 x 240 RGB565 frame needs

1,228,800 bits and fits in the FPGA BRAM.

6. VGA Timing

`vga_controller.v` is mostly counters. `hcount` runs from 0 to 799, and `vcount` runs from 0 to 524. The visible region is only 640 x 480.

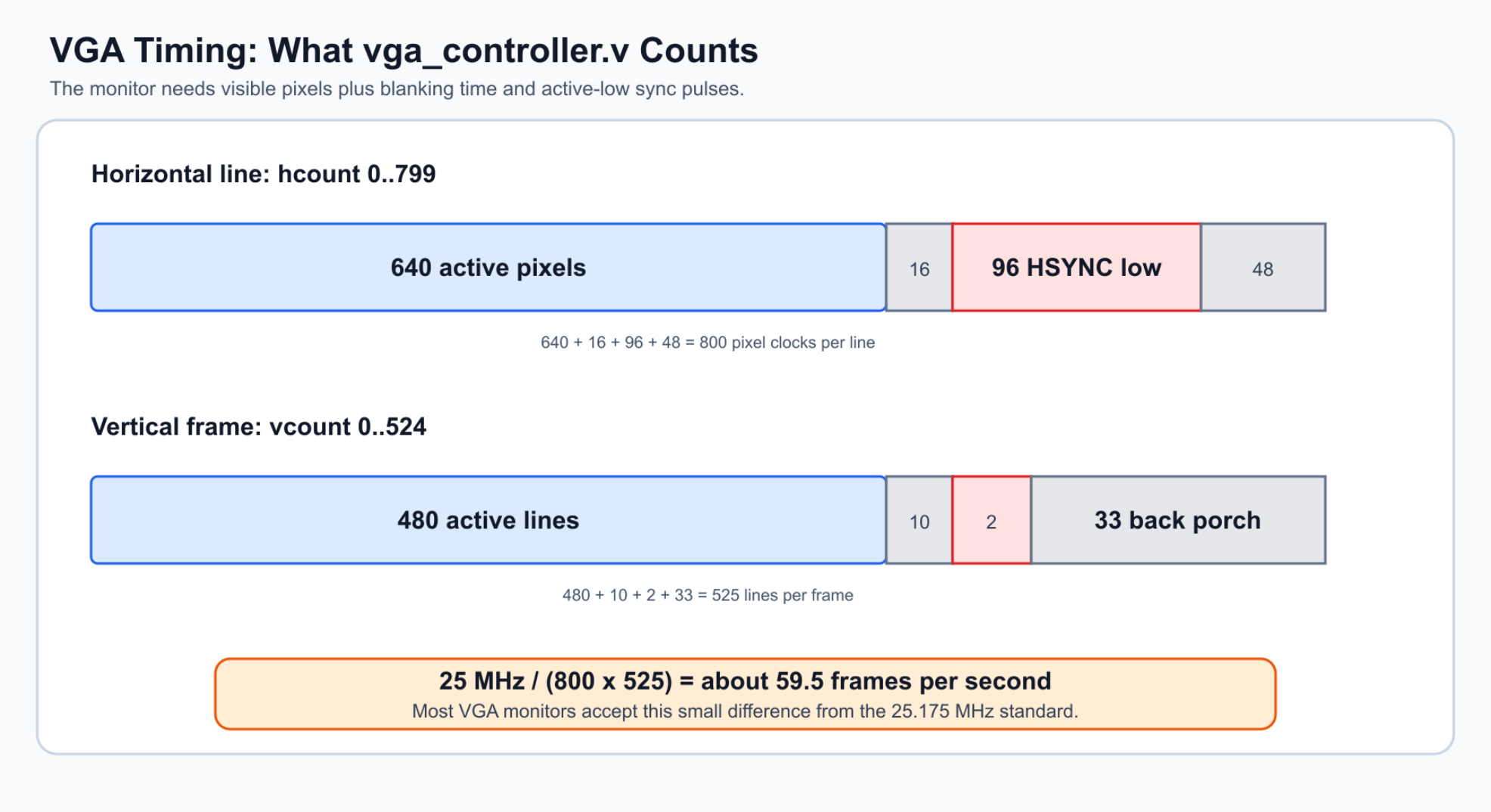

Figure 6: VGA 640 x 480 timing shown as an image, not a LaTeX chart.

The active-low sync logic is:

```
assign hsync  = ~((hcount >= H_SYNC_START) && (hcount < H_SYNC_END));
assign vsync  = ~((vcount >= V_SYNC_START) && (vcount < V_SYNC_END));
assign active = (hcount < H_ACTIVE) && (vcount < V_ACTIVE);
```

7. Display Mapping

`display_scaler.v` maps the current VGA screen coordinate to a stored BRAM pixel address. It centers a 360 x 480 image, scales by 171/256, and swaps coordinates for rotation/mirroring.

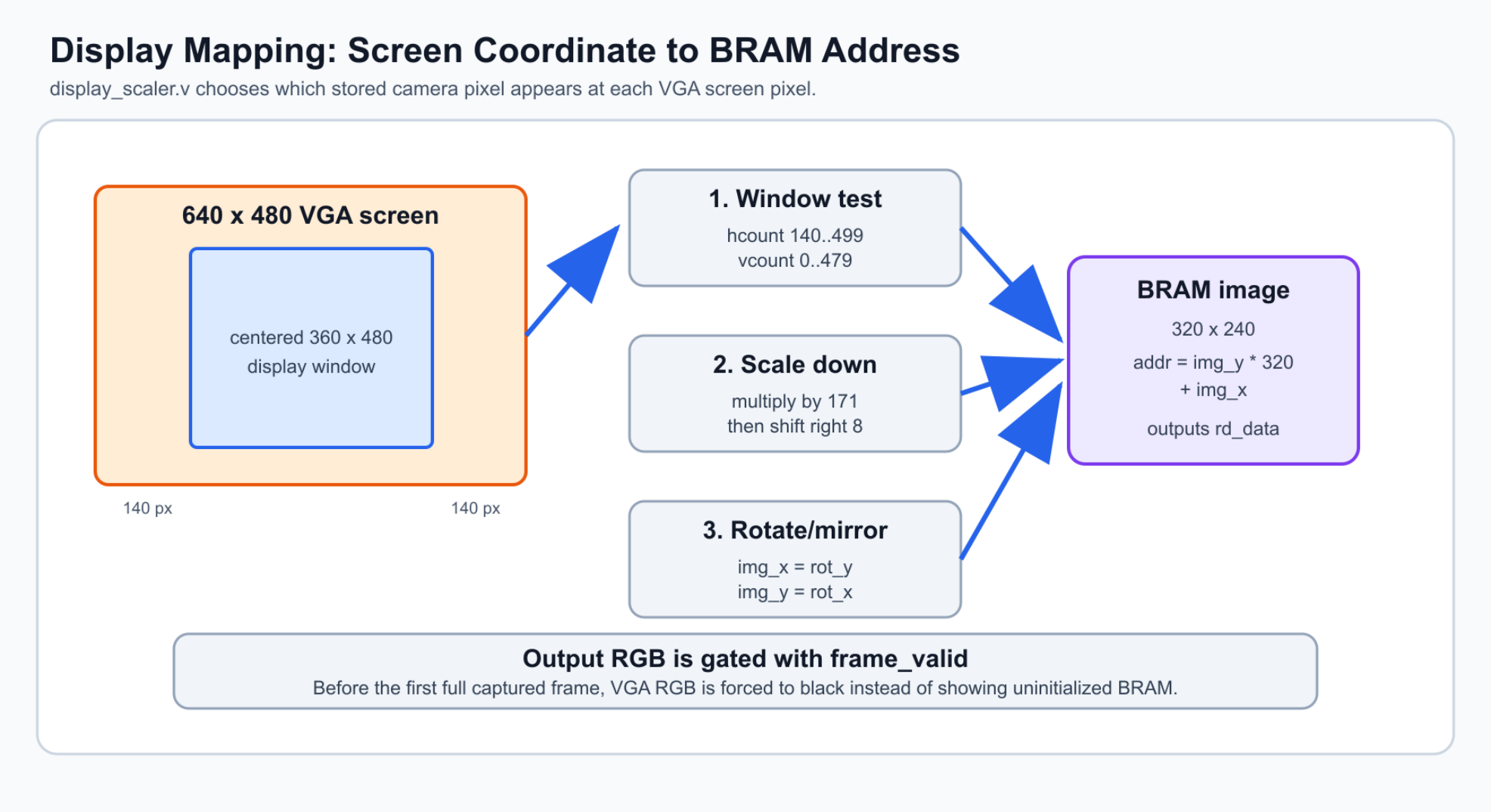

Figure 7: How display_scaler maps VGA screen coordinates to BRAM addresses.

Key mapping:

```
valid_area = hcount >= 140 && hcount < 500 && vcount < 480;
rot_x      = (screen_x * 171) >> 8;
rot_y      = (vcount   * 171) >> 8;
img_x      = rot_y;
img_y      = rot_x;
addr       = img_y * 320 + img_x;
```

8. Filter Engine

`filter_engine.v` is combinational. It takes one RGB565 pixel and outputs one RGB565 pixel. It does not need a clock and does not need a line buffer.

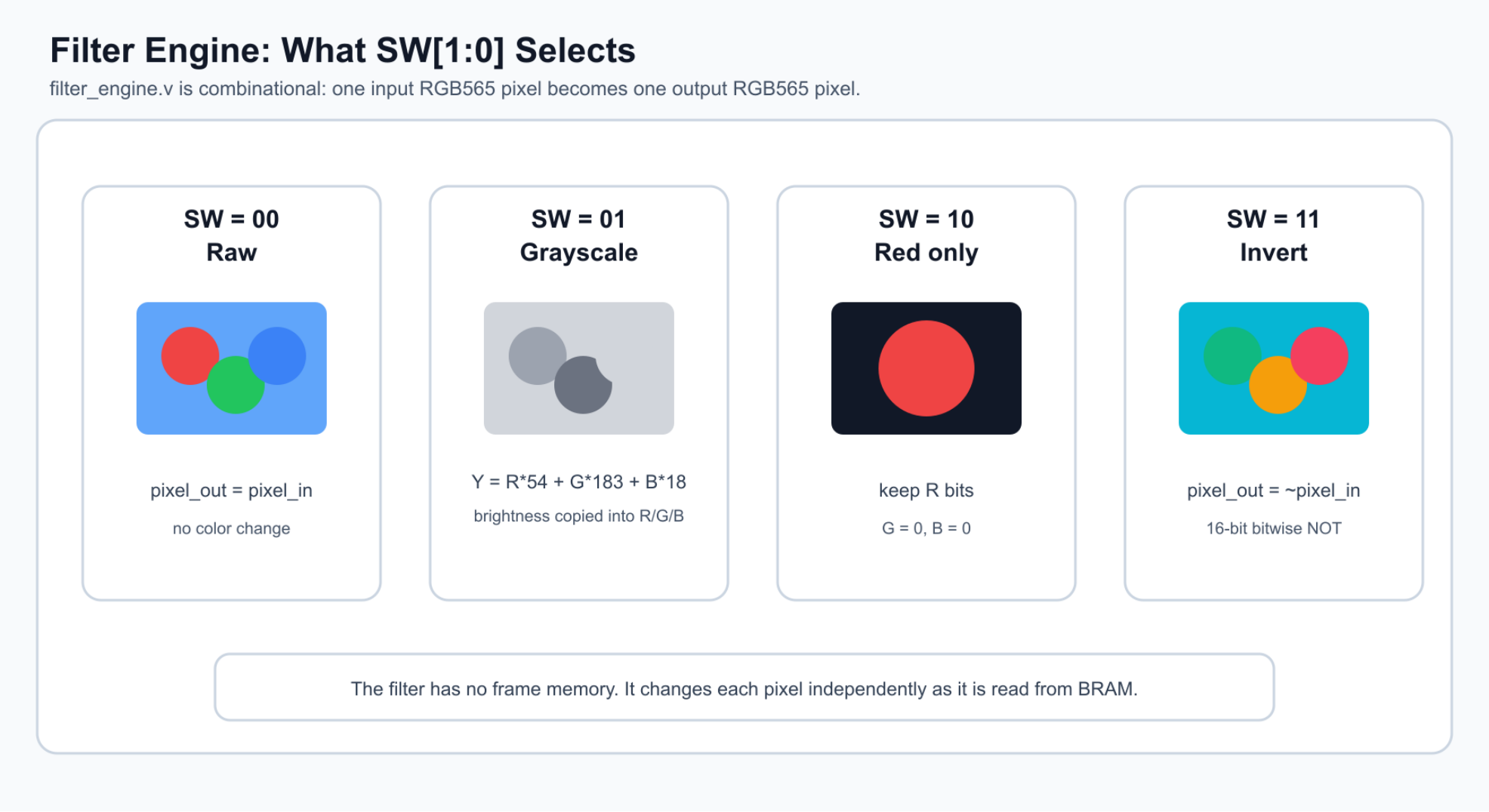

Figure 8: Switch-selected filter modes.

SW[1:0]	Mode	Operation
00	Raw	Output pixel equals input pixel.
01	Grayscale	Weighted brightness copied into R, G, and B fields.
10	Red only	Keep red bits, clear green and blue.
11	Invert	Bitwise invert the 16-bit RGB565 pixel.

9. Top-Level Module

`top.v` connects every subsystem:

- Clocking Wizard and ODDR camera clock output.
- Power-on reset and reset synchronization.
- SCCB camera configuration.
- Camera capture and BRAM write port.
- VGA controller and BRAM read port.
- Display scaler, filter engine, and sync delay.
- Frame-complete CDC through a toggle synchronizer.

Important top-level code:

```
assign cam_sda = sda_out ? 1'bz : 1'b0;

always @(posedge cam_pclk) begin
    if (cam_rst) frame_done_toggle <= 1'b0;
    else if (frame_done) frame_done_toggle <= ~frame_done_toggle;
end

wire frame_done_pix = fd_sync[2] ^ fd_sync[1];
```

10. Debug Flow

When the image does not work, debug from left to right. A later stage cannot work if an earlier stage is not producing valid signals.

Figure 9: Recommended debug order for this project.

11. What to Remember

- The camera sends bytes, not full pixels.
- Two bytes form one RGB565 pixel.
- The frame buffer is the safe handoff between camera timing and VGA timing.
- VGA timing is counter-based and must include blanking and sync periods.
- The filter engine changes one pixel at a time.
- `frame_valid` prevents garbage from appearing before the first full frame.